Lecture 6

In Class Exercises

Exercise 1

- Give a brief argument that the running time of PARTITION on a subarray of size is *n* is Θ(n).

- Modify QUICKSORT to sort into monotonically decreasing order.

Exercise 1 (solution)

- Give a brief argument that the running time of PARTITION on a subarray of size is *n* is **Θ**(n).

The **for** loop of lines 3–6 iterates $n - 1$ times, and each iteration takes $\Theta(1)$ time. The parts of the procedure outside the loop take $\Theta(1)$ time, for a total of $\Theta(n)$ time.

- Modify QUICKSORT to sort into monotonically decreasing order.

Just change the test in line 4 to $A[j] \geq x$.

Exercise 2

- Use the substitution method to prove that the recurrence $T(n) = T(n - 1) + \Theta(n)$ has the solution $T(n) = \Theta(n^2)$.

Exercise 2

- Use the substitution method to prove that the recurrence $T(n) = T(n - 1) + \Theta(n)$ has the solution $T(n) = \Theta(n^2)$.

To show that $T(n) = O(n^2)$, denote by c the constant hidden in the $\Theta(n)$ term. Guess that $T(n) \le dn^2$ for a constant d to be chosen. We have

$$\begin{aligned} T(n) &\le T(n-1) + cn \\ &\le d(n-1)^2 + cn \\ &= dn^2 - 2dn + d + cn \,. \end{aligned}$$

This last quantity is less than or equal to dn^2 if $-2dn + d + cn \le 0$, which is equivalent to $d \ge cn/(2n-1)$. This last inequality holds for all $n \ge 1$ and $d \ge c$.

For the lower bound, use the same c as for the upper bound, and guess that $T(n) \ge dn^2$ for a constant d to be chosen. Changing $\le$ to $\ge$ above yields $T(n) \ge dn^2$ if $d \le cn/(2n-1)$, which holds for all $n \ge 1$ and $d \le c/2$.

Thus, $T(n) = \Theta(n^2)$.

Exercise 3

- What value of *q* does PARTITION return when all elements in the subarray *A[p:r]* have the same value?

- Modify PARTITION so that *q = (p+r)/2* when all elements in the subarray *A[p:r]* have the same value.

```
PARTITION(A, p, r)
1  x = A[r]                          // the pivot
2  i = p − 1                         // highest index into the low side
3  for j = p to r − 1                // process each element other than the pivot
4      if A[j] ≤ x                   // does this element belong on the low side?
5          i = i + 1                 // index of a new slot in the low side
6          exchange A[i] with A[j]   // put this element there
7  exchange A[i + 1] with A[r]       // pivot goes just to the right of the low side
8  return i + 1                      // new index of the pivot
```

Exercise 3 (solution)

- What value *q* of does PARTITION return when all elements in the subarray A[p : r] have the same value?

If all the elements in subarray $A[p:r]$ have the same value, then the test in line 4 of PARTITION evaluates to true every time. When the **for** loop of lines 3–6 terminates, all elements other than the pivot will be in the subarray $A[p:i]$, where $i = r - 1$. Line 7 leaves the pivot in $A[r]$, and line 8 returns r as the pivot index. The result is unbalanced partitioning.

Exercise 3 (solution)

Modify PARTITION so that *q = (p+r)/2* when all elements in the subarray *A[p:r]* have the same value.

```
Partition(A, p, r)
    x=A[r]
    i = p − 1
    equal = True
    for j = p to r − 1
        if A[j] != x
            equal = False
        if A[j] ≤ x
            i = i + 1
            Exchange A[i] with A[j]
    Exchange A[i+1] with A[r]
    if equal is True
        return (p+r)/2
    else return i+1
```

Exercise 4

- What is the running time of Quicksort when all elements of array A have the same value?

- Write a recurrence equation for this case.

Exercise 4 (solution)

- What is the running time of Quicksort when all elements of array A have the same value?

- Write a recurrence equation for this case.

When all elements of array A have the same value, every split of partition yields a maximally unbalanced partition. The recurrence for the running time is then $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.

Exercise 5

- When RANDOMIZED-Quicksort runs, how many calls are made to the random-number generator RANDOM in the worst case? How about in the best case? Give your answer in terms of Θ-notation.

Exercise 5 (solution)

- When RANDOMIZED-Quicksort runs, how many calls are made to the random-number generator RANDOM in the worst case? How about in the best case? Give your answer in terms of Θ-notation.

In the best case, the recursion tree is as balanced as possible at every level. Thinking of a full binary tree with n leaves, each internal node represents a call of RANDOMIZED-QUICKSORT that calls RANDOMIZED-PARTITION. Since a full binary tree with n leaves has $\Theta(n)$ internal nodes, there are $\Theta(n)$ calls to RANDOM in the best case.

The worst-case recursion tree always has a split of $n-1$ to 0, so that each recursive call is on a subproblem only one element smaller. This recursion tree has $n-1$ internal nodes, so that again there are $\Theta(n)$ calls to RANDOM.